

}

UNIT 3

→ Recursion

- Function calls itself
- Factorial calculation & Fibonacci series

→ Factorial

```
#include <stdio.h>
int factorial (int n)
```

```
{
    if (n == 0)
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial (n-1);
```

```
    }
```

```
}
```

```
int main ()
```

```
{
    int num = 5;
```

```
    printf ("Factorial of %d is %d\n",
```


num, factorial (num));

return 0;

→ Fibonacci series

#include <stdio.h>

int fibonacci (int n)

{ if (n == 0) return 0;

else if (n == 1) return 1;

else return fibonacci (n-1) + fibonacci (n-2);

}

int main()

{ printf ("Enter no of terms!");

scanf ("%d" & n);

printf ("Fibonacci series :");

for (i = 0; i <= n; i++)

{ printf ("%d", fibonacci (i));

}

printf ("\n");

return 0;

}

- Structures
- stores multiple attributes of an entity
- 'struct' key work

eg →

```
struct employee
{
    int ID;
    char name [20];
    float salary;
} e1, e2;
```

declares variables of structures

ID, name, salary → members/fields of struct

→ To give info about structure

By (.)
By (>)

```
#include <stdio.h>
#include <string.h>
struct employee
{
    int ID;
    char name [50];
} e1;

int main ()
{
    e1.ID = 101;
    strcpy (e1.name, "sonoo Jaiswal");
    printf ("employee id : %d\n", e1.id);
    printf ("employee name : %s\n", e1.name);
    return 0;
}
```


→ Array of structure

```
struct Student {
    int ID;
    char name [50];
    float marks;
};
```

struct Student students [3]; Array of 3 Student

Eg:-

```
#include <stdio.h>
struct Student {
    int ID;
    char name [50];
    float marks;
};
```

```
int main () {
    struct Student students [3];
    for (int i=0; i<3; i++) {
        printf ("Enter details of student %d\n", i+1);
        printf ("ID:");
        scanf ("%d" & students [i].ID);
        printf ("Name:");
        scanf ("%s" & students [i].name);
        printf ("Marks");
        scanf ("%f" & students [i].marks);
    }
    printf ("\n Student details \n");
    for (int i=0; i<3; i++) {
        printf ("ID: %d, Name: %s, Marks: %f\n",
            students [i].id, students [i].name, students [i].marks);
    }
```

```
return 0;
```

}

→ Union

Different data types in same location

→ Union V/s Structure

In structure, each member has its own memory

In union, all members share same memory. A union can hold only one value at a time, the memory occupied is the size of its largest member.

Union — union name {

data type — member 1;

data type — member 2;

|
};

eg-1

union car {

char name [50];

int price;

} car1, car2;

eg-2

#include <stdio.h>

union union job {

char name [32];

float salary;

int worker's number;

} uJob;

struct struct Job {

char name [32]

float salary;


```

int worker's number;
}
int main() {
    printf("Size of union = %d bytes", sizeof(uJob));
    printf("Size of structure = %d bytes", sizeof(&Job));
    return 0;
}

```

Output:-
 Size of union = 32 bytes
 Size of structure = 40 bytes

eg-3 #include <stdio.h>
 union Data {

```

int i;
float f;
}

```

```

int main() {
    union Data data;
    data.i = 10;
    printf("data.i = %d\n", data.i);
    data.f = 220.5;
    printf("data.f = %d\n", data.f);
    return 0;
}

```


→ Pointer

→ stores memory address of another variable

→ declared using * ^{dereferencing operator}

→ Pointer Declaration
datatype * pointer name

→ `int * a` → pointer to an integer

→ * is used to declare pointer as well as to get the value stored at address stored in pointer

→ Pointer initialization

Initialise with address of another variable with '&' (address of) operator. This is called referencing a pointer.

pointer variable = & variable;

```
int a = 10;
printf("a/b.X = %d", *a);
```

```
int b = 5;
```

```
a = &b; printf("a/b.X = %d", *a);
```

```
#include <stdio.h>
```

```
int main()
```

```
{ int var = 10;
```

```
int * ptr = &var;
```

```
printf("%d", *ptr);
```

```
return 0;
```

```
}
```


→ #include <stdio.h>
 int main ()
 int a = 42;
 int * p = &a;
 printf ("value of a: %d\n", a);
 printf ("Address of a: %p\n", &a);
 printf ("value of (address of a)p : %p\n", p);
 printf ("value pointed to by p: %d\n", *p);
 return 0;

→ Output:
 value of a: 42
 Address of a: 0x7ffedp44
 Value of p: 0x7ffedp44
 Value pointed to by p: 42

→ int a = 43
 int *p = &a;
 printf ("%d", a);
 printf ("%p", &a);
 printf ("%p", p);
 printf ("%d", *p);

Diagram:
 a ← 43
 p ← H1028
 H1028
 H1032
 (Memory location)

Output:
 42
 H 1028
 H 1028
 42

→ Call by reference

```
#include <stdio.h>
void swap (int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main () {
    int a = 5, b = 10;
    swap (&a, &b);
    printf ("After swap: a=%d, b=%d\n", a, b);
    return 0;
}
```

Output

After swap: a=10, b=5

- Advantages of pointers
- Dynamic memory allocation & deallocation
 - Efficient accessing of arrays & structures
 - Forms complex data structures with ease
 - Reduce program length & execution time

→ Issues with pointers

- Memory corruption if incorrect value provided
- Complex to understand
- Responsible for memory leaks
- Slower than variables
- May cause segmentation fault if uninitialized